

2048 — C++ Console Game

О проекте

Консольная реализация игры 2048 на C++ с цветным выводом в терминал через ANSI коды

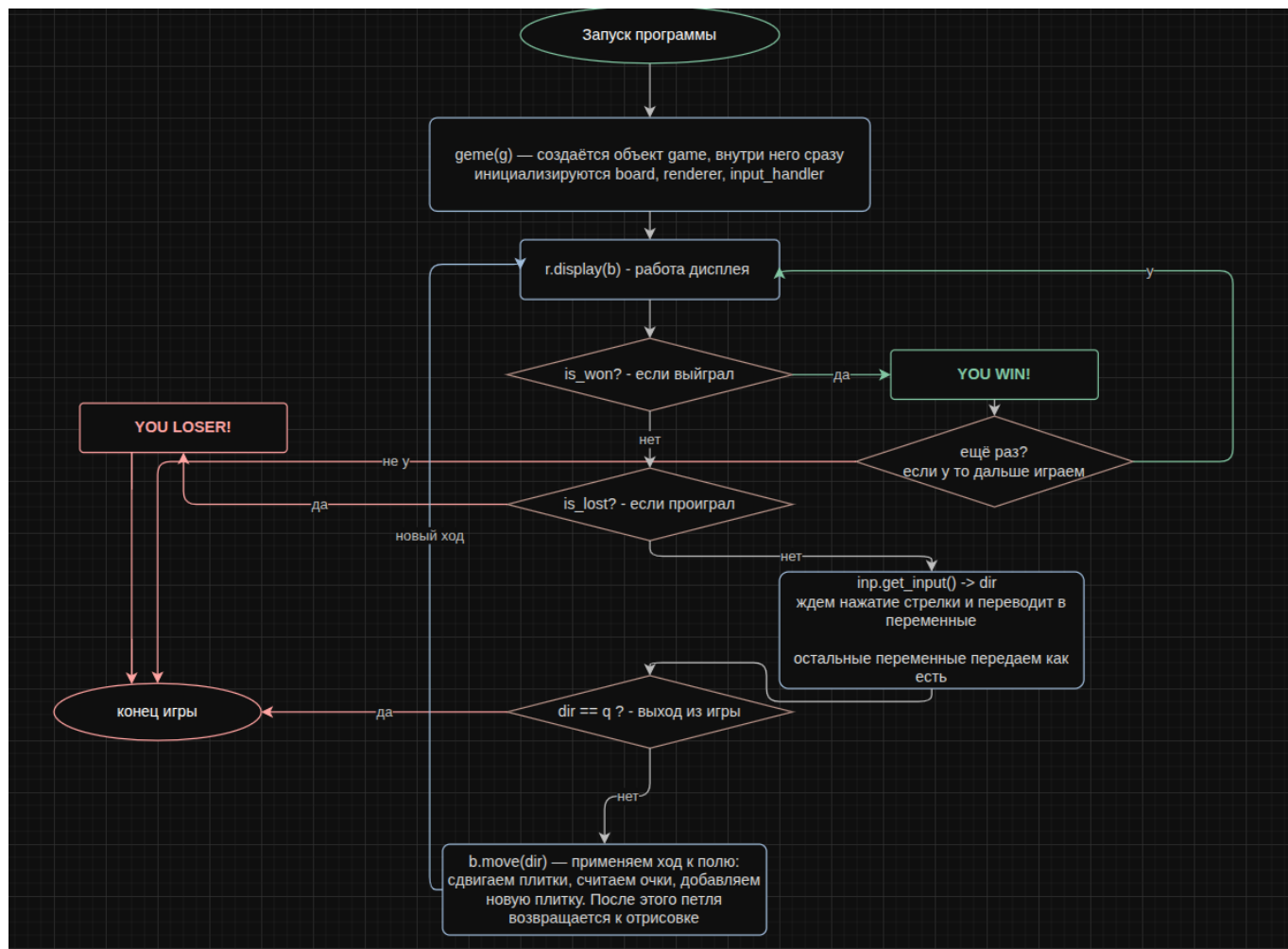
Игрок управляет стрелками на клавиатуре - плитки скользят в выбранную сторону, одинаковые сливаются и удваиваются

Цель - собрать плитку со значением 2048 (дальше можно тоже продолжить при желании играть)

Игра заканчивается когда нет ни одной пустой клетки и нет возможных слияний

Особенности реализации

- **Один алгоритм на все 4 направления.** функция `shift_line` всегда двигает плитки влево (хотя по факту она не знает про столбы и строки, но по логике индексы начинаются слева направо [0, 1, 2, 3 ...]) Для остальных направлений строка или столбец переворачивается перед подачей в функцию и разворачивается обратно после
- **Нет буферизации ввода.** я решил использовать `termios.h` для перевода терминала в сырой режим - это когда клавиша читается мгновенно без ожидания Enter После чтения терминал возвращается в нормальный режим (и настройки терминала не сбрасываются!)
- **Стрелки - это три байта.** при нажатии стрелки терминал присылает последовательность `\033 [ A/B/C/D` Тут я использовал `getchar`, который читает по одному байту и переводит в символы `u/d/l/r`



Структура проекта

```

game_2048/
├── images
│   ├── functional_model_code.png
│   └── Object_model_code.png
├── include
│   ├── board.h
│   ├── color.h
│   ├── game.h
│   ├── input.h
│   └── renderer.h
├── main.cpp
├── Makefile
├── program
├── README.md
├── src
│   ├── board.cpp
│   ├── color.cpp
│   ├── game.cpp
│   └── input.cpp

```

```
└─ renderer.cpp
```

Сборка и запуск

```
make          # сборка
make run      # сборка и запуск
make clean    # удалить бинарник
```

Классы

board

Сам по себе класс хранит состояние игрового поля и содержит всю игровую логику

Переменные:

- `grid[4][4]` - двумерный массив 4×4, хранит значения всех плиток Ноль означает пустую клетку
- `score` - текущий счёт, который должен увеличиваться на значение новой плитки каждый раз когда происходит слияние

Приватные методы:

- `get_empty_cells()` - проходит по всему полю и возвращает вектор координат `{row, col}` всех клеток где стоит ноль!
- `add_random_tile()` - берёт случайную пустую клетку и ставит туда 2 - 90% или 4 - 10%
- `shift_line(line)` - ключевой алгоритм, который принимает строку из 4 чисел, убирает нули, сливает соседние одинаковые значения (левый поглощает правый, увеличивается вдвое), добавляет нулями до длины 4 и возвращает результат

Публичные методы:

- `board()` - конструктор, который все обнуляет и ставит две стартовые плитки
- `get_cell(i, j)` - возвращает значение клетки по координатам Он по идее нужен классу `renderer` для доступа к приватному массиву `grid`
- `getscore()` - возвращает текущий счёт
- `move(dir)` - применяет ход в направлении `l/r/u/d` далее сохраняет старое состояние поля, вызывает `shift_line` для каждой строки или столбца в нужном порядке, сравнивает с сохранённым, если поле изменилось добавляем клетку
- `is_won()` - проверяет есть ли на поле хоть одна клетка со значением 2048

- `is_lost()` - проверяет нет ли пустых клеток и нет ли соседних одинаковых по горизонтали и вертикали Если всё так => ходов нет и игра проиграна
-

renderer

отвечает только за вывод поля в терминал

Методы:

- `display(b)` - очищает экран командой `\033[2J\033[H`, выводит заголовок и счёт, затем рисует сетку 4×4 Для каждой непустой клетки запрашивает цвет у `color::get()`, центрирует число внутри ячейки шириной 7 символов и выводит с цветом
-

color

Один статический метод в виде полей соответствия к непустым клеткам

Методы:

- `get(val)` — по значению плитки возвращает строку с ANSI кодом нужного цвета Вызывается как `color::get(val)` просто как утилита
-

input_handler

Отвечает за чтение одной клавиши с клавиатуры в обход стандартной буферизации

Методы:

- `get_input()` - сохраняет текущие настройки терминала, переключает его в сырой режим (отключает `ICANON` - режим, который ждем enter прежде чем отправить и `ECHO` - режим, который показывает, что ты написал на экране), читает символ если пришло `\033`, то `getchar` читает ещё два байта и переводит букву `A/B/C/D` в символ `u/d/l/r`, затем по идее восстанавливает терминал (сырой режим живёт ровно пока читается одна клавиша) и возвращает символ
-

game

главный управляющий класс Он владеет всеми остальными объектами и управляет ходом игры

Переменные:

- `b` - объект `board`, хранит поле и логику
- `r` - объект `renderer`, рисует поле
- `inp` - объект `input_handler`, читает ввод

Методы:

- `run()` — бесконечный цикл: рисует поле => проверяет победу => проверяет проигрыш => читает клавишу => применяет ход Выходит из цикла при победе (если игрок не хочет

продолжать), проигрыше или нажатии **q**

